DB 이전 · 실시간 API 연동 안내

현재 구현 기준 | 확인일 2026.11.09 (한국 시간) | 비밀번호·실제 인증 키 미포함

1. 서버 환경과 현재 공개 상태

항목	현재 상태
DB 서버 PC	Windows / 내부 IP 192.168.0.167
MySQL	8.4.13 / 포트 3306 / DB nodeverse
MongoDB	8.0.34 / 포트 27017 / DB nodeverse_logs
Django API·대시보드	개발 PC 192.168.0.29에서 실행 / 현재 127.0.0.1:8000만 수신
외부 연동 상태	수신 API 구현 완료. LAN 공개 주소 및 테스트 키는 아직 준비 전

DB 서버 주소와 API 서버 주소는 다릅니다. 192.168.0.167:3306 또는 :27017은 DB 접속용이며 HTTP API 주소가 아닙니다.

2. 수신 API 주소 / HTTP 방식

용도	현재 전체 주소	방식
회원 정보 수신	http://127.0.0.1:8000/api/game/player-login	POST
게임 기록 수신	http://127.0.0.1:8000/api/game/records	POST

끝 슬래시(/)는 붙이지 않습니다. 위 localhost 주소는 API 실행 PC에서만 사용할 수 있습니다. 외부 PC용 주소는 바인딩·허용 호스트·방화벽 설정 후 별도로 확정합니다.

3. 인증과 테스트 키

```
Authorization: Bearer <GAME_SERVER_API_KEY>
Content-Type: application/json
```

현재 키는 미발급·미설정 상태입니다. 키가 비어 있으면 503, 설정된 키와 요청 키가 다르면 401을 반환합니다. 키 설정 후 수신 서버 재시작이 필요합니다.

운영자가 다음 명령으로 키를 생성해 양쪽 서버의 비밀 환경 변수에 설정합니다. 발급 키와 비밀번호는 별도 경로로 전달하며 Unity·브라우저 클라이언트에는 넣지 않습니다.

```
.\.venv\Scripts\python.exe -c "import secrets; print(secrets.token_urlsafe(48))"
```

관리자 로그인 쿠키·CSRF 토큰은 게임 서버 수신 API에 사용하지 않습니다. 운영 공개 시 HTTPS를 적용합니다.

회원 정보 수신

게임 서버가 로그인 검증을 완료한 뒤 호출합니다. 게임 비밀번호를 여기서 검증하거나 저장하지 않습니다.

1. 요청 JSON

```
{
  "member_id": "test-member-001",
  "display_name": "Test Player"
}
```

필드	필수	검증 규칙
member_id	예	게임 서버의 변경되지 않는 고유 회원 ID. 문자열 1~120자. 영문·숫자·_-.@ 허용
display_name	예	표시 이름 1~80자. 공백만 있는 값·제어 문자 불가

정확히 두 필드만 전달합니다. 전체 본문은 4KB 이하입니다. 닉네임이나 변경 가능한 로그인명을 member_id로 사용하지 않습니다. MySQL collation에 따라 대소문자가 동일시될 수 있으므로 대소문자만 다른 회원 ID는 발급하지 않습니다.

2. 성공 응답

```
{
  "player": {
    "player_id": "22222222-2222-4222-8222-222222222222",
    "member_id": "test-member-001",
    "display_name": "Test Player",
    "source": "live"
  },
  "created": true
}
```

최초 등록은 HTTP 201, 기존 회원은 HTTP 200입니다. 재전송 시 같은 player_id를 반환하고 표시 이름을 갱신합니다. 관리자 계정이나 웹 인증 세션은 만들지 않습니다.

3. 회원 ID 연결 규칙

식별자	용도
요청 member_id	게임 서버의 원본 회원 ID
응답 player_id	대시보드가 발급한 UUID. 이후 세션의 player_id에 사용
MySQL member_id	응답 player_id와 같은 UUID. DB에는 CHAR(32)로 저장
external_member_key	game:member:<게임 서버 원본 회원 ID> 형태의 외부 연결 키

예시 UUID는 설명용입니다. 실제 연동에서는 회원 수신 응답값과 송신 측에서 생성한 기록 UUID를 사용합니다.

게임 기록 수신 · 생성과 종료

POST /api/game/records는 기록 종류(record_type)와 데이터(data)를 가진 JSON 객체를 받습니다.

1. 세션 시작 예시

```
{
  "record_type": "session",
  "data": {
    "id": "11111111-1111-4111-8111-111111111111",
    "player_id": "22222222-2222-4222-8222-222222222222",
    "topic": "stack",
    "started_at": "2026-11-09T10:00:00+09:00"
  }
}
```

2. 기록 종류와 필수 필드

record_type	data 필수 필드 / 규칙
session	id, player_id, topic, started_at. 선택: level, game_version
problem	id, task_id(문제 코드), content_version, title, topic
run	id, session_id, task_id(problem의 UUID), started_at
attempt	id, run_id, number(1 이상), started_at
event	id, session_id, sequence(1 이상), event_type, occurred_at. 구조화된 이벤트는 run_id, 시도·연산 이벤트는 attempt_id도 필수
end_attempt	id, status(success / failure / abandoned), ended_at
end_run	id, status(completed / abandoned), ended_at
end_session	id, status(completed / abandoned), ended_at

topic은 stack / queue / tree입니다. session·run·attempt는 진행 중 상태로 생성하며 별도 종료 요청으로 마칩니다. 타임스탬프는 Z 또는 +09:00 같은 시간대가 포함된 ISO 8601 값을 전달합니다.

3. 순서와 재시도

회원 수신 → problem 등록 → session 시작 → run 시작 → attempt 시작 → event 전송 → end_attempt → 문제 종료 이벤트 전송 → end_run → end_session 순으로 호출합니다.

문제 정의는 (task_id, content_version)별 UUID를 고정합니다. 같은 요청을 재시도할 때는 UUID·sequence·본문 내용을 유지합니다. 같은 ID의 다른 내용이나 중복 순서는 오류입니다.

주의: problem.task_id는 사람이 읽는 문제 코드이고 run.task_id는 해당 problem의 UUID입니다. 게임 서버 원본 회원 ID 대신 회원 수신 응답의 player_id를 사용합니다.

행동 이벤트 · 응답과 검증

게임 서버가 인증된 회원의 세션·문제·시도를 확인한 뒤 전송합니다. 전체 로그 요청은 64KB 이하입니다.

1. 연산 성공 이벤트 예시

```
{
  "record_type": "event",
  "data": {
    "id": "55555555-5555-4555-8555-555555555555",
    "session_id": "11111111-1111-4111-8111-111111111111",
    "run_id": "33333333-3333-4333-8333-333333333333",
    "attempt_id": "44444444-4444-4444-8444-444444444444",
    "sequence": 3,
    "event_type": "operation_succeeded",
    "operation": "pop",
    "occurred_at": "2026-11-09T10:00:10+09:00",
    "state_before": {"size": 2},
    "state_after": {"size": 1},
    "payload": {}
  }
}
```

세션·문제 실행·풀이 시도가 먼저 존재해야 합니다. 연산 실패(operation_failed)는 operation과 failure_reason이 필요하고, 구조화된 hint는 hint_id와 1 이상의 hint_step이 필요합니다. received_at은 수신 서버가 생성합니다.

2. 성공·종료·실패 응답

```
201 / 200: {"record_id": "<UUID>", "created": true}
200 retry: {"record_id": "<UUID>", "created": false}
200 final: {"record_id": "<UUID>", "status": "finalized"}
401: {"error": "invalid_server_credentials"}
400: {"error": "invalid_or_conflicting_record"}
503: {"error": "game_integration_disabled"}
503: {"error": "log_storage_unavailable"}
```

HTTP	의미
400 / 409	검증·관계·기록 충돌 / 회원 외부 ID가 데모 레코드와 충돌(회원 API)
401 / 405	인증 키 누락·불일치 / 지원하지 않는 HTTP 방식
413 / 415	본문 크기 초과 / application/json 필요
503	키 미설정 또는 MongoDB 저장소 장애

네트워크 실패·5xx는 같은 ID로 재시도합니다. 400은 입력 오류나 처리 중인 세션 여부를 확인합니다. MySQL 연결 장애가 항상 위 JSON 형식으로 반환되는 것은 아닙니다.

MySQL 테이블 · 회원 ID 정의

대상 DB: nodeverse / UUID 값·비밀번호 해시·권한 관계를 보존하며 읽기 쉬운 물리 이름으로 구성했습니다.

1. 게임 회원: game_members

컬럼	자료형	설명
member_id	CHAR(32), PK	대시보드 UUID. API에서는 하이픈 포함 36자리 문자열
external_member_key	VARCHAR(150), UNIQUE	game:member:<원본 회원 ID>
nickname	VARCHAR(80)	회원 표시 이름
data_source	VARCHAR(8)	CHECK: live 또는 demo
registered_at	DATETIME(6)	대시보드 최초 등록 시각

2. 관리자: admin_accounts

컬럼	자료형 / 의미
admin_id	CHAR(32), PK / 관리자 UUID
login_id / password_hash	VARCHAR(150), UNIQUE / VARCHAR(128), 비밀번호 해시
nickname / account_role	VARCHAR(80) / VARCHAR(7), CHECK: admin 또는 과거 learner
created_at / updated_at	DATETIME(6) / 계정 생성·최종 수정 시각
last_login_at	DATETIME(6), NULL 허용 / 마지막 로그인
is_active / is_staff / is_superuser	TINYINT(1) / 활성·Django 관리 표식·전체 권한
email / first_name / last_name	VARCHAR(254) / VARCHAR(150) / VARCHAR(150)

3. 권한 및 Django 내부 테이블

테이블	주요 컬럼과 역할
admin_group_memberships	id, user_id → admin_accounts.admin_id, group_id → auth_group.id
admin_permission_assignments	id, user_id → admin_accounts.admin_id, permission_id → auth_permission.id
auth_group / auth_group_permissions	그룹: id, name / 연결: id, group_id, permission_id
auth_permission	id, name, codename, content_type_id → django_content_type.id
django_content_type	id, app_label, model / 권한에 사용하는 모델 종류
django_session	session_key, session_data, expire_date / 관리자 웹 로그인 유지
django_migrations	id, app, name, applied / DB 구조 변경 적용 이력

회원은 관리자 계정과 분리됩니다. django_session은 게임 플레이 세션이 아닙니다. SQL 테이블 10개가 유지되며 게임 로그 테이블은 MongoDB 이전 후 제거했습니다.

MongoDB 컬렉션 · 인덱스

대상 DB: nodeverse_logs / 모든 _id는 UUID 문자열이며 UTC BSON Date로 시각을 저장합니다.

1. 컬렉션과 주요 필드

컬렉션	주요 내용
game_sessions	_id, player_id, topic, level, game_version, started_at, ended_at, status, duration_ms
game_problems	_id, task_id(문제 코드), content_version, title, topic
problem_runs	_id, session_id, task_id(문제 UUID), started_at, ended_at, status, active_ms, paused_ms
solution_attempts	_id, run_id, number, started_at, ended_at, outcome
gameplay_events	_id, session_id, run_id, attempt_id, sequence, event_type, occurred_at, received_at, operation, failure_reason, state_before, state_after, hint_id, hint_step, payload

2. 실제 적용된 인덱스

컬렉션	추가 고유 인덱스	조회 인덱스
game_sessions	-	(player_id, started_at) (topic, started_at)
game_problems	(task_id, content_version)	-
problem_runs	-	(session_id, started_at) (task_id, started_at)
solution_attempts	(run_id, number)	-
gameplay_events	(session_id, sequence)	(run_id, occurred_at) (occurred_at, event_type)

모든 컬렉션에 기본 _id 고유 인덱스가 추가로 있습니다. 복합 조회 인덱스는 모두 오름차순입니다.

3. 검증 규칙과 저장 경계

MongoDB 자체 JSON Schema validator는 현재 미설정입니다. 수신 API에서 UUID·자료구조·상태·참조 관계·시간 범위·이벤트별 필수값을 검증합니다. 컬렉션에 직접 쓰면 이 검증을 우회합니다.

state_before, state_after, payload는 JSON 객체이며 각각 16KB 이하입니다. password, password_hash, authorization, session_cookie, secret_key, admin_registration_code 키는 중첩 구조에서도 차단합니다. 그 외 비밀값도 송신 측에서 제외해야 합니다.

MySQL member_id와 MongoDB game_sessions.player_id를 연결합니다. MongoDB에는 DB 외래 키가 없으므로 서버가 관계를 확인합니다. 여러 컬렉션과 MySQL을 묶는 원자적 트랜잭션은 없으며, 세션 단위 잠금과 동일 ID 재시도로 중간 실패를 처리합니다.

실행 위치 · 이전 시험 · 공동 검증

현재 구현 경로와 이후 배치 결정을 구분합니다. API 공개와 키 발급을 마친 뒤 테스트 회원으로 검증합니다.

1. 현재 실행 위치와 미확정 사항

구성 요소	현재 위치 / 상태
MySQL·MongoDB	서버 PC 192.168.0.167에서 실행
Django 수집 API·대시보드	개발 PC 192.168.0.29에서 실행. 현재 localhost 전용
Kafka·Spark	현재 실시간 수신 경로에 배포·연결되어 있지 않음
서버 PC로 API 이전	같은 PC에서 실행할지 별도 확정 필요

현재 경로: 게임 서버 → Django 수신 API → MySQL 회원 / MongoDB 기록 → 대시보드 조회. Kafka·Spark를 포함하는 최종 파이프라인 배치 여부는 팀과 별도로 확정합니다.

2. MongoDB 7.0.16 / nodeverse → 8.0.34 / nodeverse_logs

바로 덮어쓰지 않고 임시 DB에서 먼저 이전 시험을 진행합니다. 원본 컬렉션이 현재 스키마와 같다고 가정하지 않으며, 회원 매핑·ID 자료형·날짜·상태·필드 의미를 먼저 비교합니다.

단계	확인 내용
원본 조사	컬렉션별 건수, 샘플 구조, 인덱스·validator, 회원 ID와 날짜 자료형 확인
임시 DB 시험	백업을 보존하고 별도 시험 DB에 복원. 현재 수집 API가 사용하는 대상 DB는 덮어쓰지 않음
변환·연결	원본 회원 ID → 대시보드 UUID 매핑. 관계 UUID와 문제 코드·문제 UUID 구분
검증·반영	건수·중복·관계·시간대·지표 비교 후 대상 DB 반영 방식과 중단 시간 확정

3. 테스트 회원으로 함께 확인할 흐름

① 게임 서버 로그인 검증 → ② 회원 수신 및 player_id 확인 → ③ 문제·세션·풀이 시도 생성 → ④ 행동 이벤트 및 종료 요청 → ⑤ MySQL 회원·MongoDB 로그 저장 확인 → ⑥ 대시보드·이벤트 목록·차트·CSV 조회 확인

같은 요청 재전송 시 중복되지 않는지, 잘못된 참조·인증 키가 거부되는지, 시간대가 올바른지도 확인합니다. 시험 데이터는 실제 데이터와 구분하고 시험 후 필요한 범위만 정리합니다.

4. 연동 전에 남은 준비

API의 LAN 바인딩·ALLOWED_HOSTS·방화벽 설정 및 접속 시험, 테스트 키 발급·양쪽 환경 변수 설정, API 최종 실행 위치 확정이 필요합니다. 현재 키가 비어 있으므로 아직 외부 게임 서버의 실시간 수신을 시작한 상태는 아닙니다.

비밀번호·실제 인증 키는 이 PDF에 포함하지 않았습니다. 상세 구현 기준: telemetry/game_api.py, telemetry/mongo.py, telemetry/recording.py 및 Django 마이그레이션. DB 구조는 2026.11.09 확인 시점 기준입니다.